AI ENGINEERING / EVALUATION / TESTING

WORKFLOW EVALUATION HARNESS

12

PROPOSED RESEARCH / NOT YET VERIFIED

THE ENGINEERING QUESTION

Can a repeatable replay harness with regression gates catch harmful AI workflow changes before they reach a live process?

PROBLEM DEFINITION AND RESEARCH CONTRIBUTION

A reusable test bench that replays synthetic workflow cases through an AI assisted pipeline and scores accuracy, latency, cost and failure modes before any release.

- Cases are synthetic or openly licensed and each carries a known expected outcome plus a labelled failure category.
- Gates compare a candidate against the current baseline. A named person reviews every failed gate and decides whether to release.
- AI assisted and never AI executed. The harness informs the release decision and never makes it.

ARCHITECTURE AND IMPLEMENTATION SPECIFICATION

1 STAGE **Case library** → 2 STAGE **Replay runner** → 3 STAGE **Scorers** → 4 STAGE **Release gate**

- Python runner that replays versioned cases against pinned prompts and model settings with seeded repeat trials.
- Scorers for field accuracy and abstention behaviour plus latency percentiles and token cost per case.
- Failure taxonomy and regression gate rules stored as configuration so thresholds are reviewed like code.

ENGINEERING DECISIONS TO MAKE EXPLICIT

The design review must record data contracts, state transitions, failure handling and why the selected baseline is defensible. Every subsystem should have an interface specification and a reproducible test route.

DESIGN BOUNDARIES AND ASSUMPTIONS

⚠ Synthetic cases cannot cover every live input so a passing gate lowers risk without removing it.

⚠ Outputs vary between runs so scores need repeat trials and stated uncertainty.

ⓘ Scope is proposed unless explicitly identified as documented existing work. All measured claims require reproducible evidence.

WORKFLOW EVALUATION HARNESS / PROTOCOL / ACCEPTANCE / REVIEW

EXPERIMENTS, VERIFICATION & PUBLICATION

12

PROPOSED RESEARCH / NOT YET VERIFIED

EXPERIMENTAL METHODOLOGY

01 Build a labelled case library with edge cases and adversarial inputs and plant known defects in candidate versions.

02 Run baseline and candidate pipelines with repeat trials and compare accuracy and latency with stated intervals.

03 Tune gate thresholds to see what share of planted defects they block and how often they block good changes.

MEASUREMENT AND ACCEPTANCE EVIDENCE

- Regression detection rate on deliberately injected defects
- Run to run score variance on identical cases
- Cost and latency per case with confidence intervals

REQUIRED REPOSITORY ARTEFACTS

- Versioned synthetic case library
- Replay runner and scorer reference implementation
- Gate rules and regression report template

TECHNICAL REVIEW QUESTIONS

Which assumptions could invalidate the result? How do the baselines behave under the same conditions? What is the strongest negative case? Which variables, seeds, workloads, software versions and machine properties must be retained for independent reproduction?

VERIFICATION MAPPING

For each acceptance claim, record the test identifier, input configuration, observed result, pass/fail definition and an independent rerun procedure. Failed or inconclusive cases remain in the report rather than being silently excluded.

PUBLICATION DISCIPLINE

Release original source, configuration files, test harnesses, raw measurements and scripts where rights permit. Distinguish simulation outcomes from physical validation and completed achievements from planned evaluation. Results must not be inferred from targets.